What makes an RNN different?

Choose one answer, then click to reveal the explanation.

Words arrive one at a time. Which basic model can pass a hidden state from one word to the next?

A A feedforward fully connected network, by itself.

B A standard CNN evaluating each word independently.

C An RNN that carries its state through the sequence.

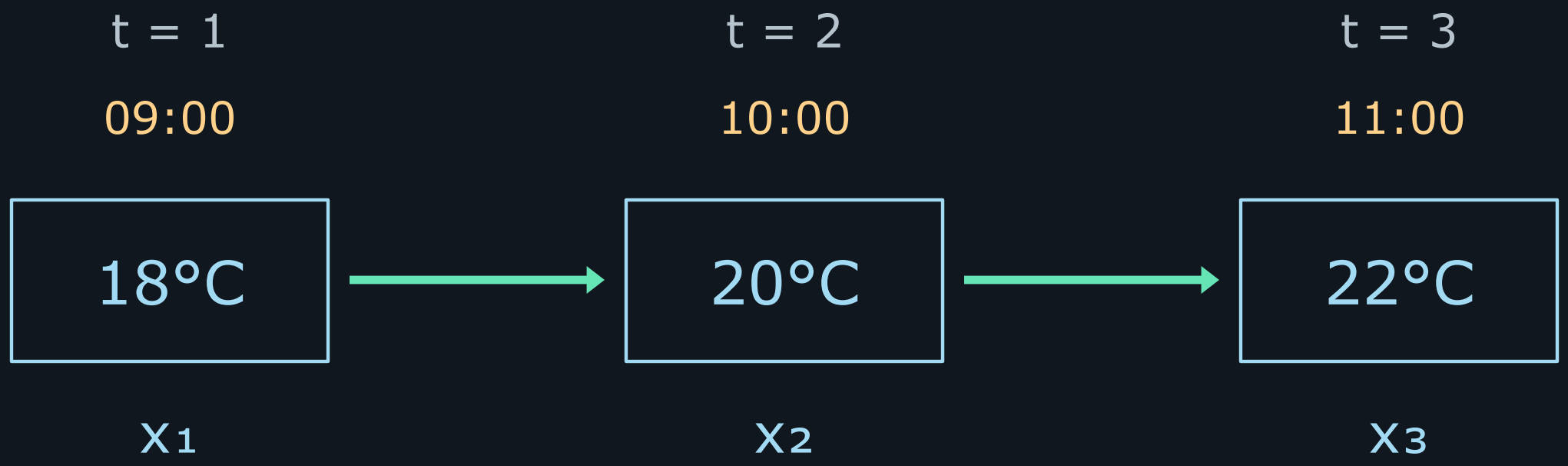
D Any model, simply because its weights have been trained.

Answer: C

An RNN updates a hidden state as it reads the sequence.

What is a sequence?

A sequence is an ordered collection of items. Here, each item is a reading.



T = 3 means three items in this sequence.

The index t tells us which item we are processing.

Does an FC network remember the last example?

Choose one answer, then click to reveal the explanation.

A feedforward fully connected (FC) network has fixed weights. What determines its output for the current input vector?

A Only the example that it processed immediately before.

B The supplied input vector and the learned weights.

C A hidden state automatically passed between examples.

D The position of this example in the evaluation list.

Answer: B

The input vector may include earlier measurements if we supply them.

How does a CNN differ from an FC network?

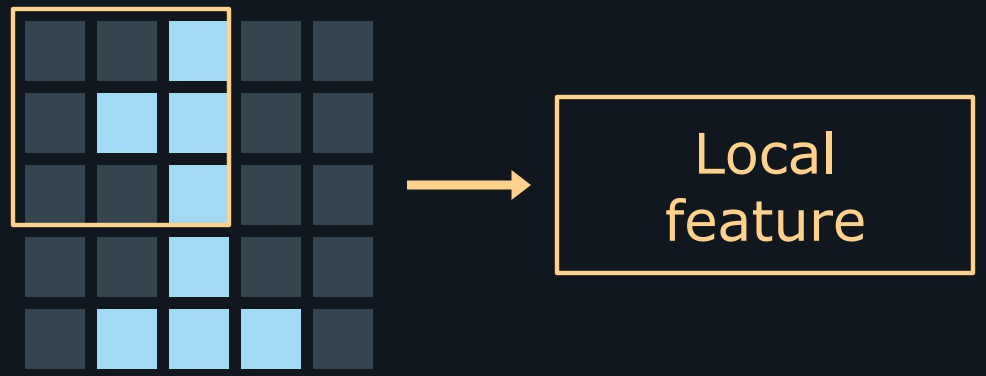
Both learn weights. They connect and reuse those weights differently.

Fully connected layer



Each output combines all input features.

Convolutional layer



One filter scans local regions.

The same filter weights are reused.

Neither basic feedforward model carries a recurrent state between images.

Does a CNN remember the previous image?

Choose one answer, then click to reveal the explanation.

**A standard CNN classifies two independent photos.
At inference, what does it use for the second prediction?**

A A recurrent state saved from the first photo.

B Its current photo and its learned weights.

C New weights learned from the first photo.

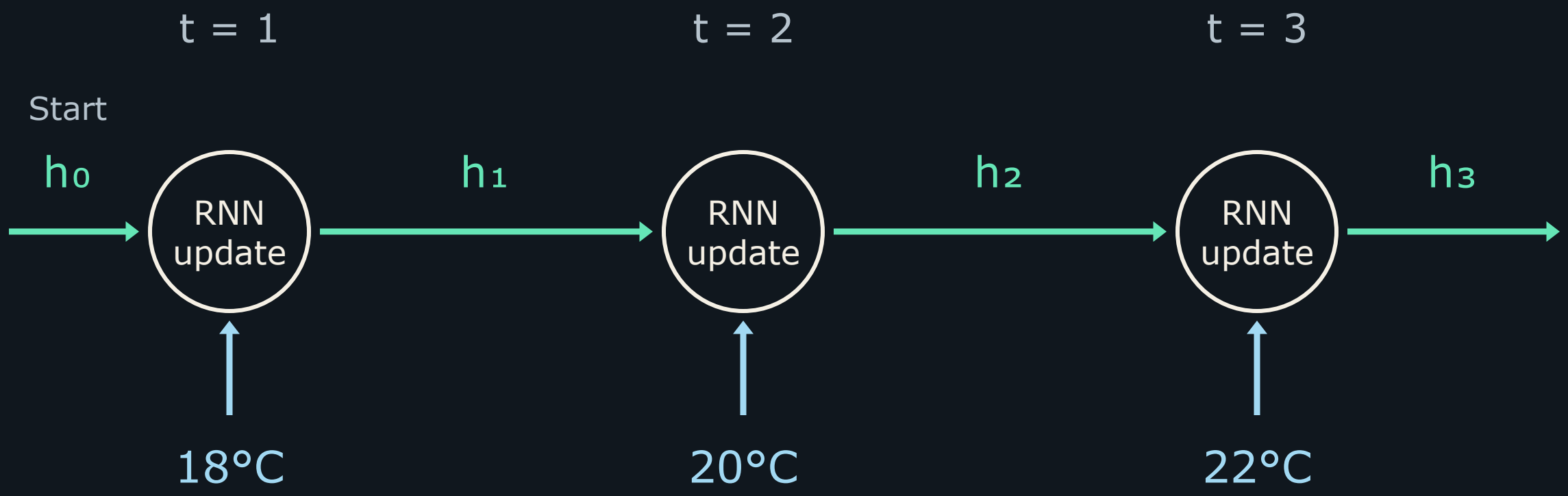
D Every photo that has ever been evaluated.

Answer: B

Learned weights persist. A standard CNN adds no image-to-image state.

How does an RNN move through time?

RNN means recurrent neural network. The update repeats for each item.

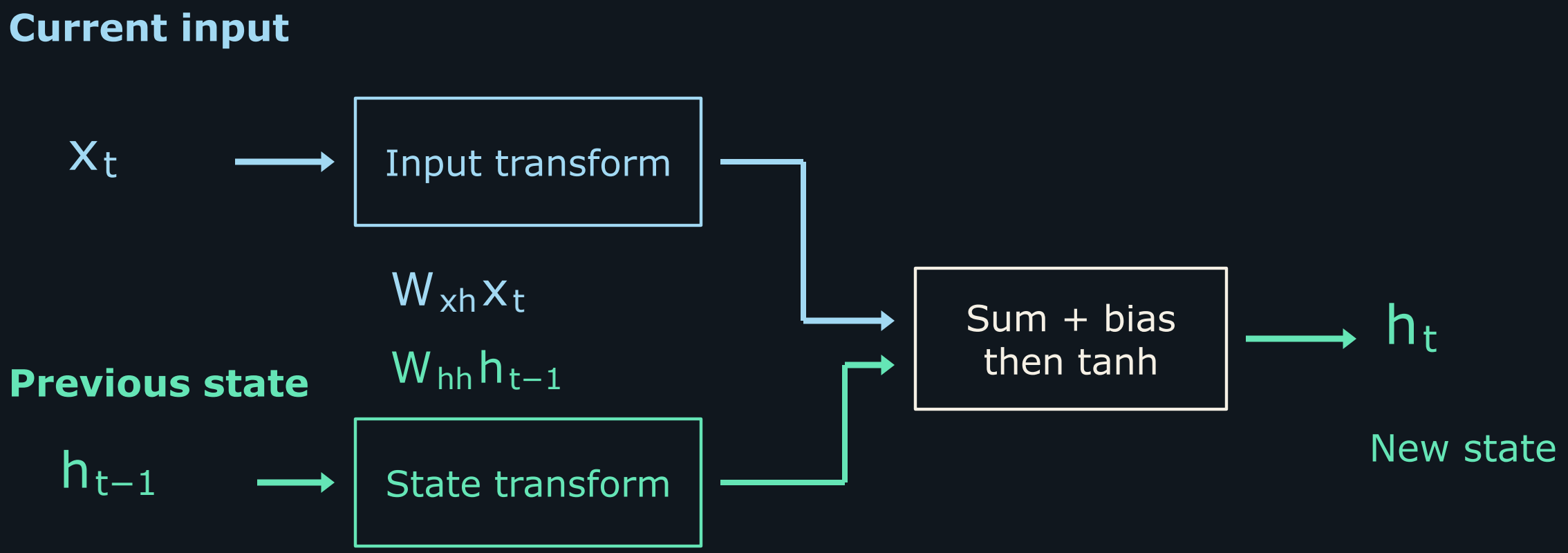


h is the hidden state: the summary carried to the next step.

The same learned update is reused at every time step.

Can an RNN use fully connected calculations?

Yes. A basic RNN combines the current input with the previous state.



Two dense transforms combine now and before. The same weights recur.

Do three steps mean three sets of weights?

Choose one answer, then click to reveal the explanation.

We draw the same RNN cell at t = 1, 2, and 3.
What changes as it reads the sequence?

A Every step receives its own independently learned weights.

B Input and state values can change. Cell weights are reused.

C The previous state is discarded before every step.

D The hidden state grows by one component at each step.

Answer: B

Unfolding shows repeated calculations using the same learned cell.

How big is the hidden state?

We choose how many values the state contains.

Readings so far

18°C, 20°C, 22°C



$h_3 =$

$$\begin{bmatrix} 0.2 \\ -0.4 \end{bmatrix}$$

Example state values

Hidden size H = 2

Useful context can remain in a small state.

The state keeps the same width when the sequence becomes longer.

Sequence length T and hidden size H are different choices.

Does a longer sequence mean a bigger model?

T counts items. Each item causes one update in a standard RNN.

T = 3

3 items

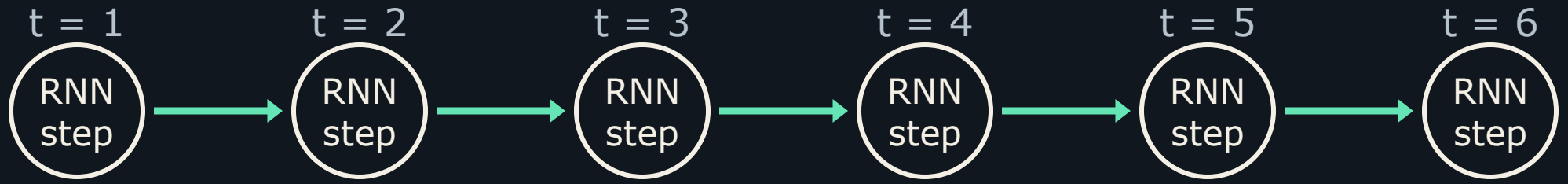


3 updates

One for each item

T = 6

6 items



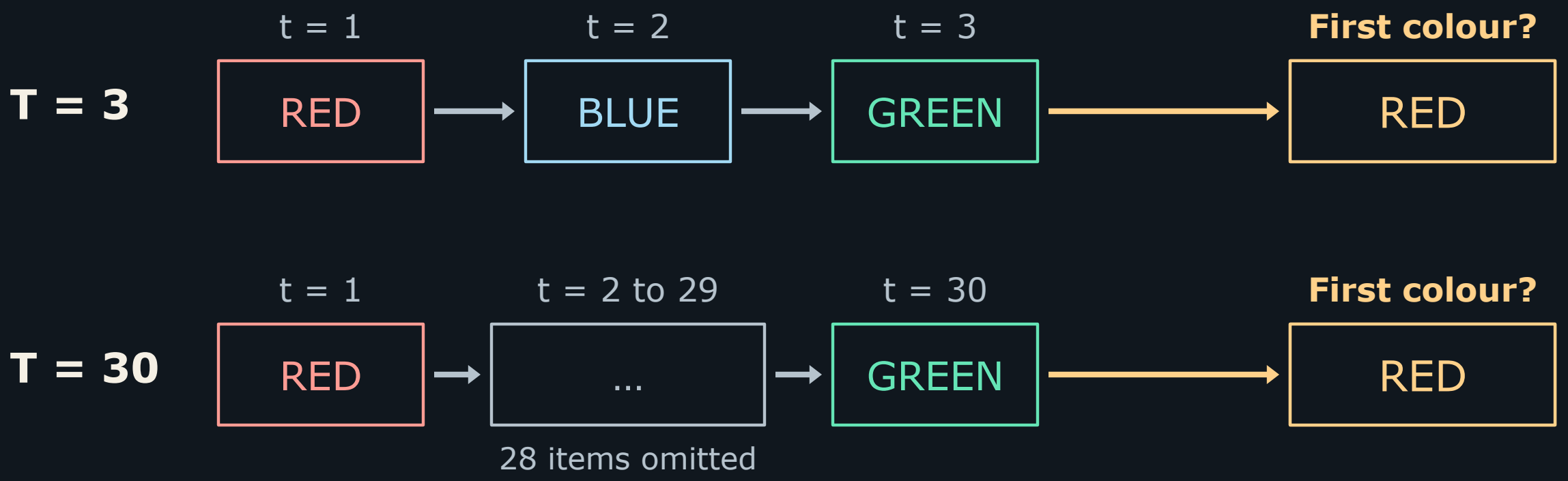
T = 6 performs twice as many updates as T = 3.

Same learned weights. Same hidden-state size.

The same rule extends to T = 30: 30 items, 30 updates.

Why can a longer sequence be harder?

Task: remember the first colour, then report it at the end.

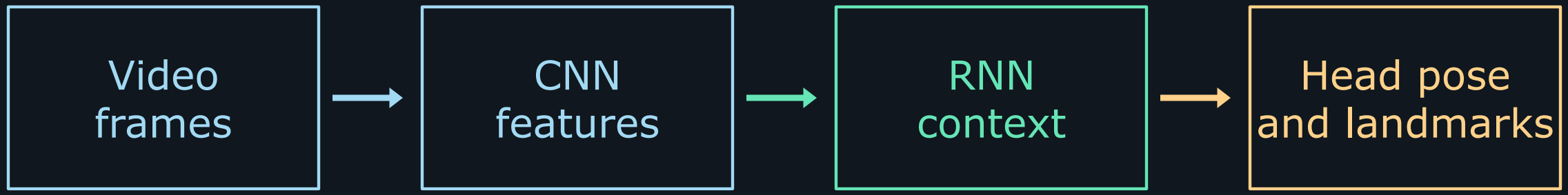


More intervening updates can make early information harder to use.

Full-sequence training usually needs more computer memory.

Can CNNs and RNNs work together?

NVIDIA combined image features with recurrent context for facial tracking.



The CNN describes each frame.

The recurrent state connects information across frames.

A sequence of frames provides context for tracking motion.

NVIDIA, AI Co-Pilot: RNNs for Dynamic Facial Analysis (2017)

NVIDIA application demo: AI Co-Pilot

2017 demonstration. About 70 seconds. Click the video in Slide Show mode.



Watch for

Face tracking
Head tracking

How can earlier frames help the current estimate?

Open NVIDIA clip online

The documented RNN component uses context across video frames.

Video: NVIDIA Newsroom. Technical example: NVIDIA facial analysis research (2017).

What does IBM’s RNN example show?

The Power of Recurrent Neural Networks (RNN). About 8 minutes.



Discuss your observations before continuing.

While you watch

What does the speaker mean by “memory”?

What moves from one time step to the next?

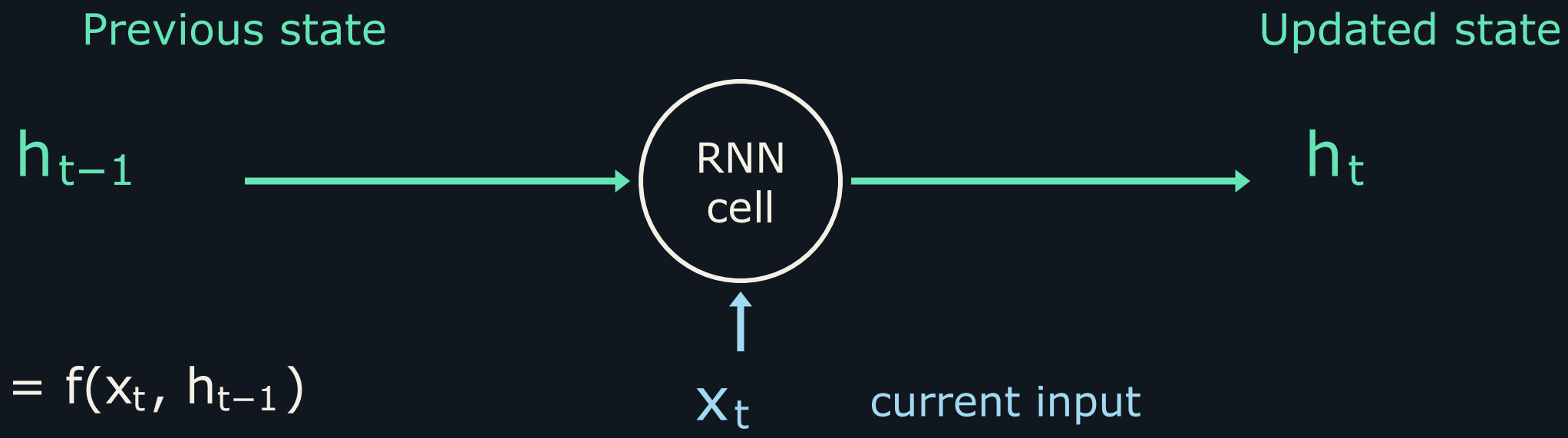
Watch the IBM video

Open IBM mirror

Internet required for this video

What does the video mean by memory?

The hidden state carries context from earlier inputs in the same sequence.



$h_t = f(x_t, h_{t-1})$

Learned weights

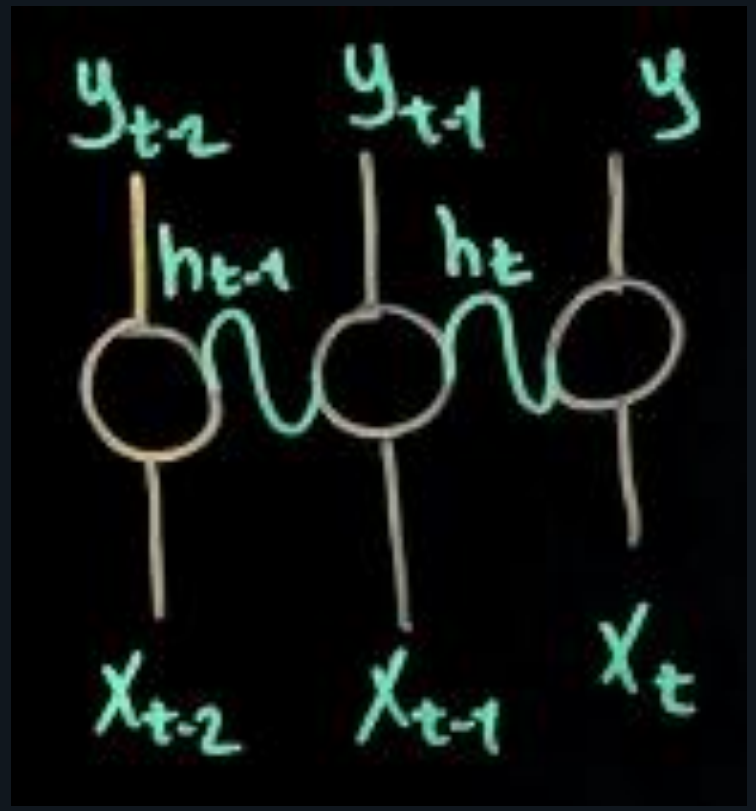
Shared across steps.
Updated during training.

Sequence state

Updated as inputs arrive.
Reset for independent sequences.

Which labels need correction?

Look only at the unrolled sketch in this screenshot. Which step produced each state?



What should be relabeled?

A The use of a previous hidden state.

B The state labels and the final y label.

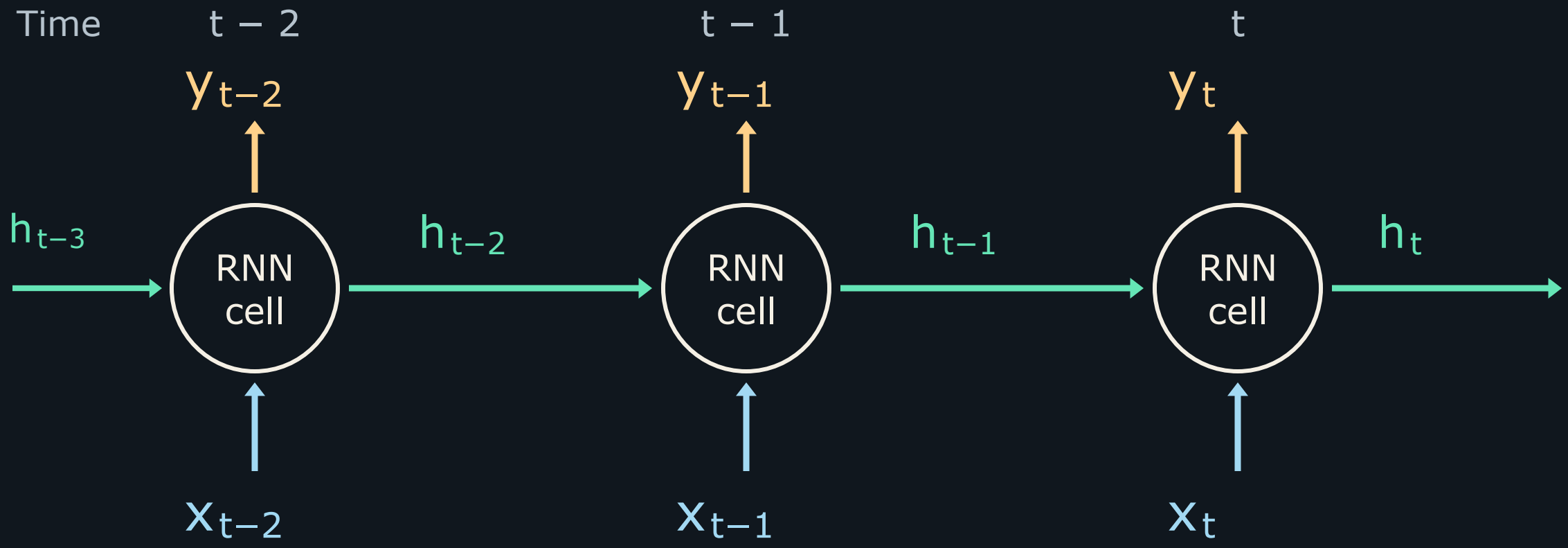
C The left-to-right order of the inputs.

D The presence of an output at each step.

Answer: B Each state takes the time index of the cell that produces it. Add t to the last y. Use clear arrows for direction.

Corrected unrolled RNN

The recurrence is correct. The connecting state labels need matching time indices.



The circle computes a state vector. The arrow carries that vector forward.

Is h_t the circle or the connection?

Choose one answer, then click to reveal the explanation.

In our diagram, what is h_t?

A The outline of the circle.

B The arrow or loop itself.

C A state vector computed by the cell and carried by the connection.

D The learned weights shared by all time steps.

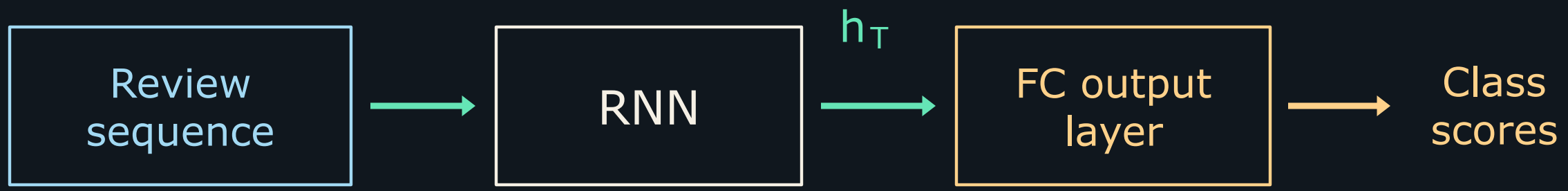
Answer: C

h_t is a vector. A legend tells us what the circle represents.
A recurrent loop is a connection that carries state.

Can a dense layer read the RNN’s state?

Yes. The RNN forms a state. A fully connected output layer reads it.

Example: classify a whole review after its last word.



$$z = W_{hy} h_T + b_y$$

The FC layer reads the state. Recurrence provides the sequence context.

What happens when we add an FC output layer?

Choose one answer, then click to reveal the explanation.

An RNN reads a review. An FC layer then predicts its sentiment. Which description is correct?

A The FC layer removes the RNN’s earlier context.

B The RNN keeps context, and the FC layer maps state to scores.

C The FC layer must also have a recurrent loop.

D The combination can process only one input word.

Answer: B

The components have different jobs and can be trained together.

Must every input step produce an output?

Choose one answer, then click to reveal the explanation.

An RNN reads “This film is great.”
How many predictions can the task ask us to produce?

A Always one prediction for each of the four words.

B Exactly one prediction, regardless of the task.

C One for the review or one per word, depending on the task.

D No predictions until all possible sentences are known.

Answer: C

The task determines which hidden states the output head reads.

What can go in, and what can come out?

“One” and “many” count items or sequence positions.

Pattern	Application
One-to-one	Image classification
Many-to-one	Review sentiment
One-to-many	Image captioning
Aligned many-to-many	One tag for every word
Many-to-many, encoder–decoder	Translation

One image or one feature vector can contain many numeric values.

One-to-one: image classification

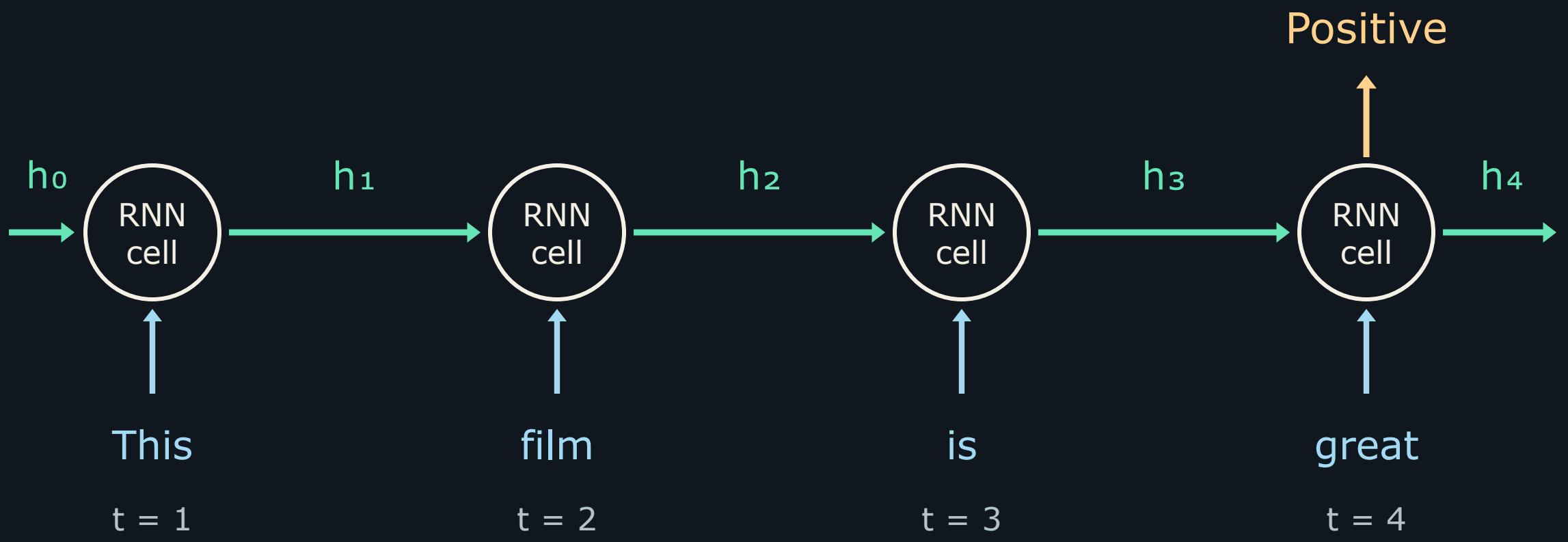
One input image produces one class prediction.



Many pixels form one input item. One class is predicted for that item.

Many-to-one: review sentiment

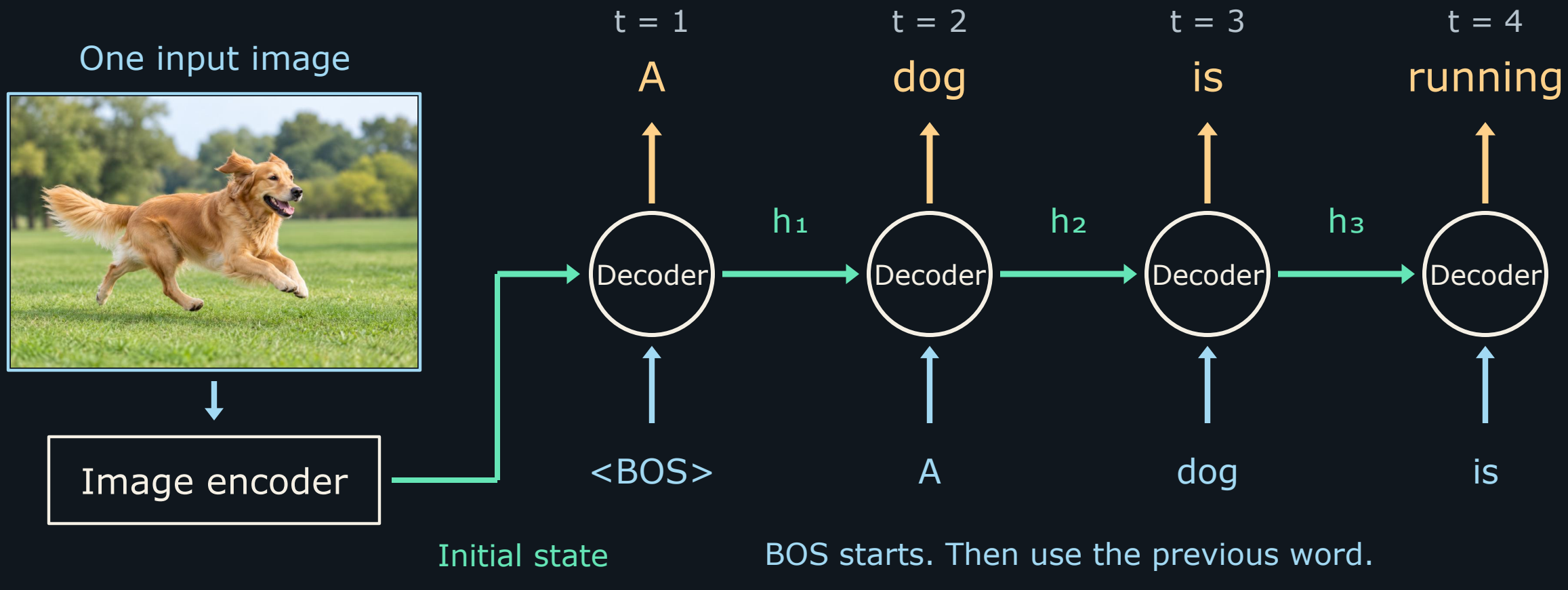
Read the complete review, then predict one sentiment label.



Four input words produce one prediction for the whole review.

One-to-many: image captioning

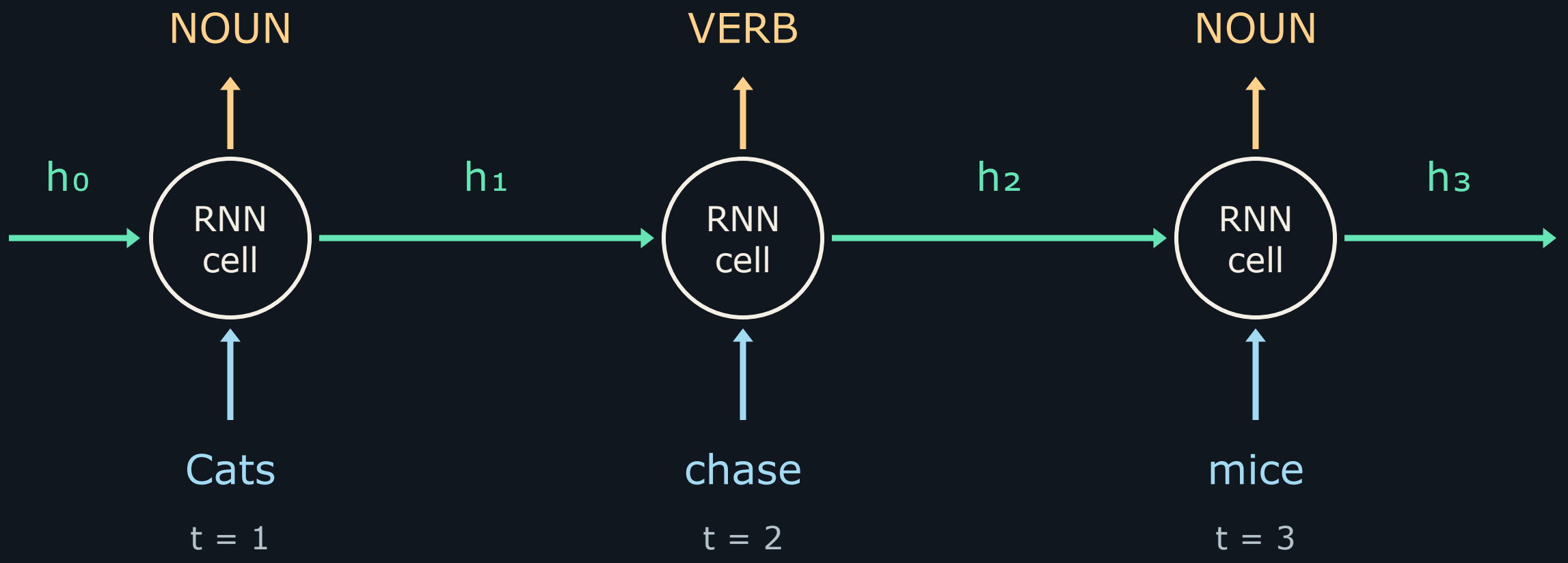
One image conditions a caption, one word at a time.



Caption: “A dog is running.” One image, four output words.

Aligned many-to-many: tag every word

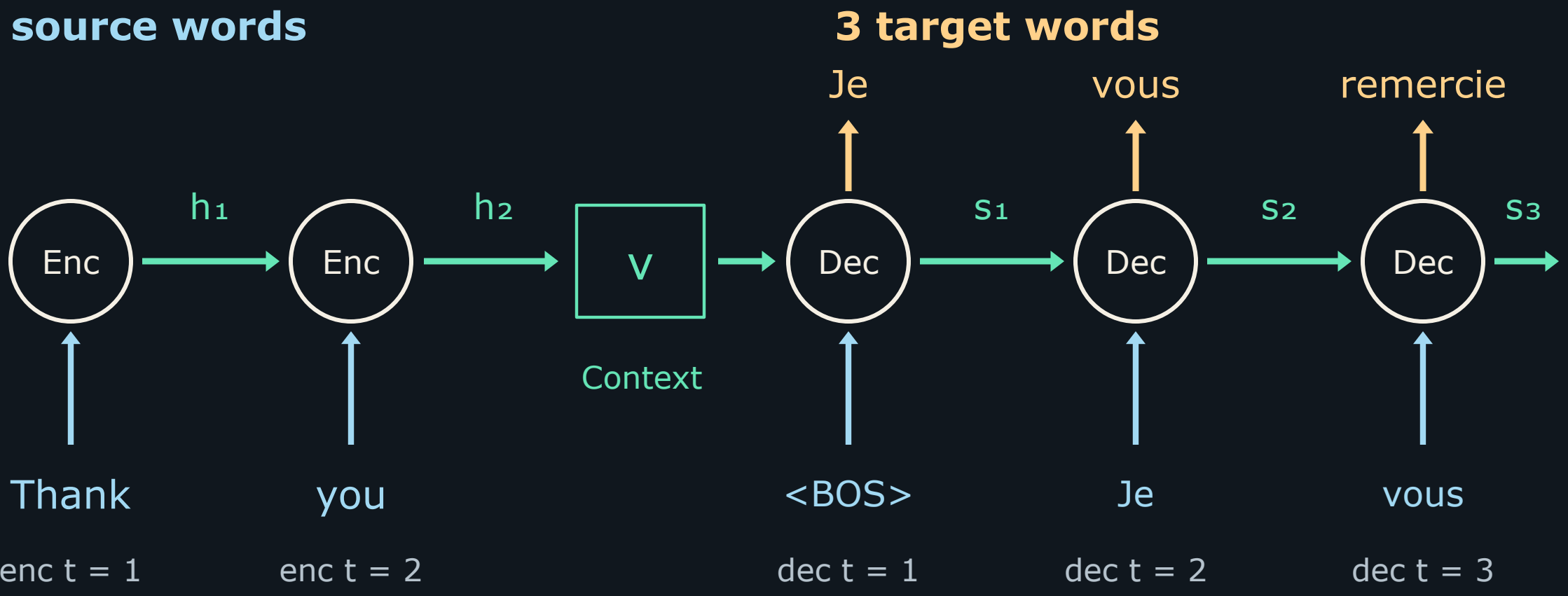
Each input position has a corresponding output position.



Three words produce three tags, aligned position by position.

Many-to-many: translation

An encoder reads the source. A decoder generates the translation.



Encoder–decoder allows different input and output lengths.

Check: Which sequence pattern fits?

Choose one answer, then click to reveal the explanation.

A model assigns one activity label to every video frame. Which pattern is this?

A One-to-many: one item generates the entire sequence.

B Many-to-one: the whole video receives only one label.

C Aligned many-to-many: each frame receives a label.

D One-to-one: the sequence is a single scalar.

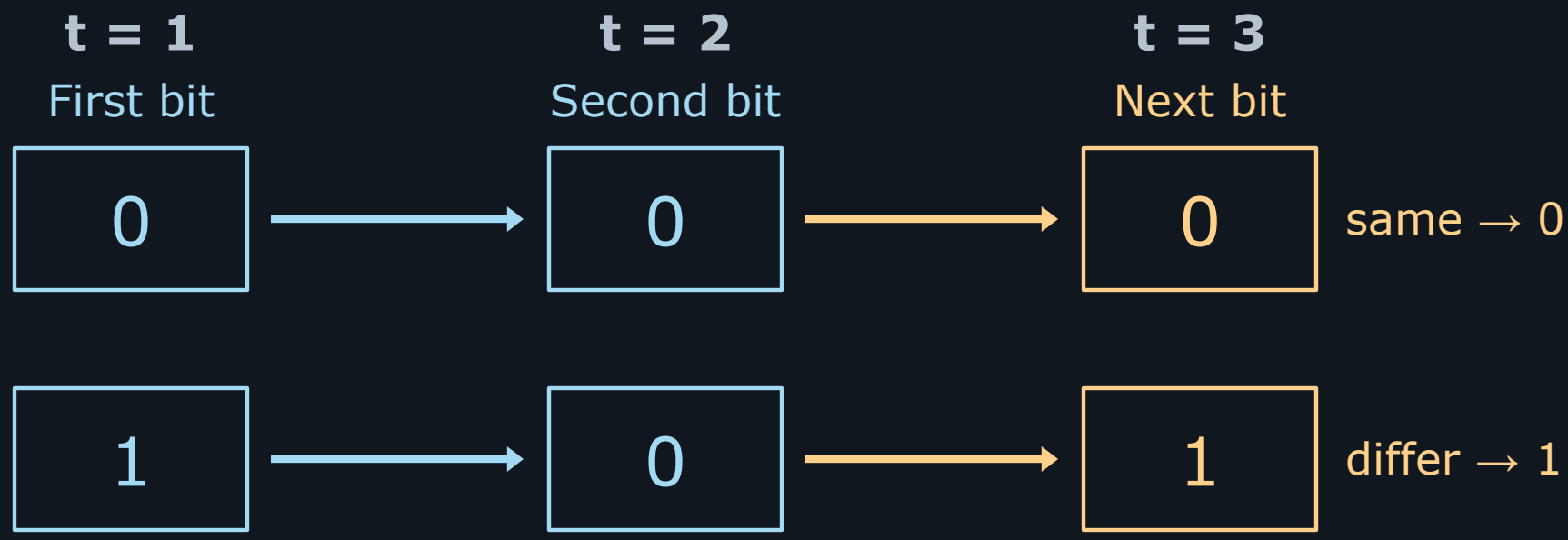
Answer: C

The input frames and their output labels align position by position.

When is the current input not enough?

Read one bit at a time. Predict the next bit.

XOR rule Same bits → 0 Different bits → 1



After the second bit, use both earlier bits to predict the third.

The current input can hide different histories

At t = 2, both examples contain 0. Their next bits must be different.

Feedforward: current bit only

Same $x_2 = 0$ in both histories.

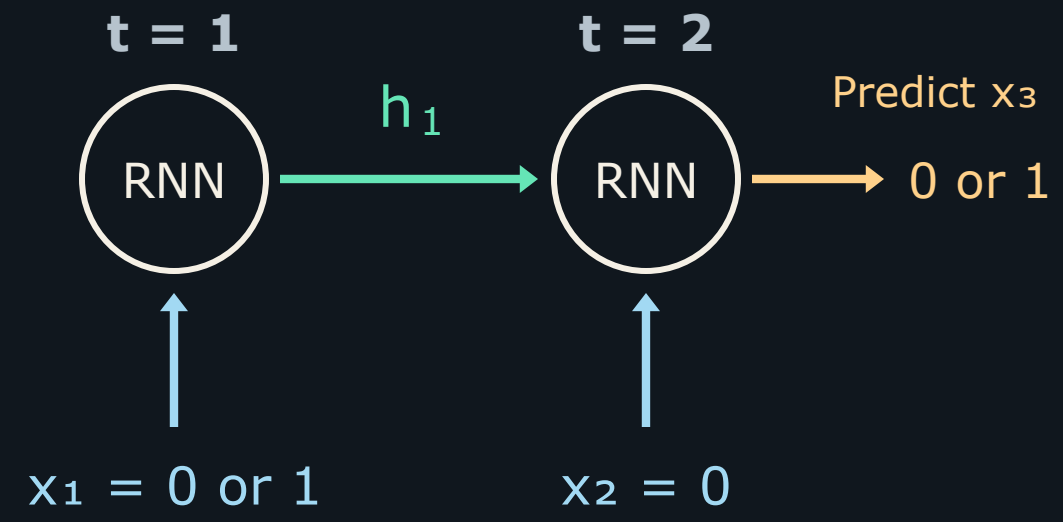


Same input → same output.

Earlier 0: need 0. Earlier 1: need 1.

x_2 alone cannot resolve both cases.

RNN: input + previous state



The state can preserve the earlier bit.

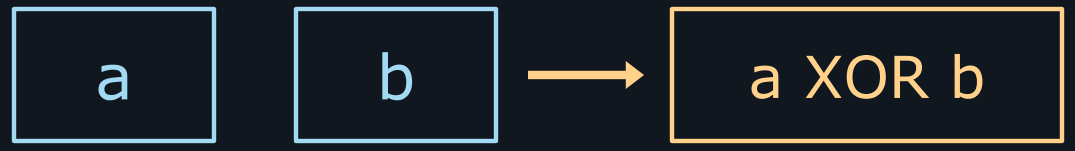
The model needs access to history when history changes the answer.

Paper finding: the RNN learned a temporal rule

Some next bits follow a rule. Other next bits are chosen randomly.

Predictable XOR bit

The two earlier bits determine the next XOR bit.



Reported: lower prediction error.

Randomly chosen bit

Past inputs cannot determine a new random bit.



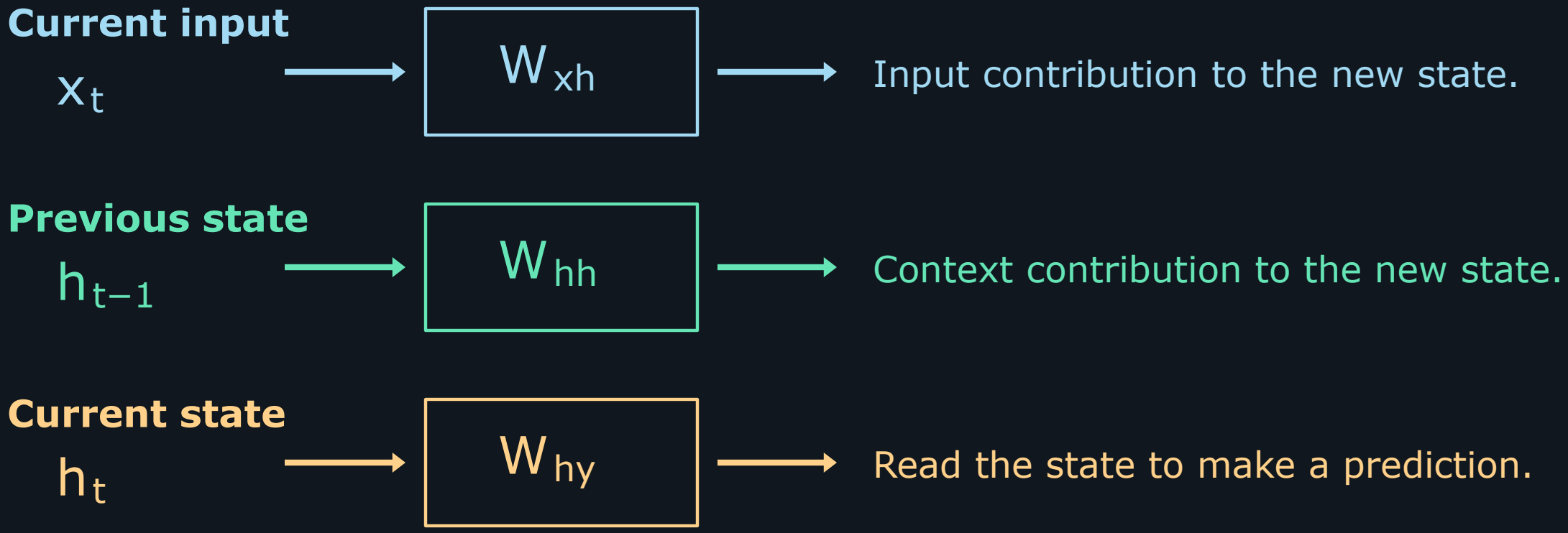
Prediction error remains higher.

Fair comparison: a nonlinear feedforward network can compute XOR when its input includes both bits $[x_1, x_2]$.

History can be supplied as input or carried in a recurrent state.

Which weights do the three jobs?

The subscripts name the source and destination.



W_{xh} and W_{hh} update the state. W_{hy} reads the new state.

The hidden-state update

Combine the current input and previous state, then apply an activation.

Input term **Earlier context** **Bias**

$$h_t = f(\; W_{xh}\, x_t \;+\; W_{hh}\, h_{t-1} \;+\; b_h \;)$$

Worked example: detect two consecutive 1s.

$$h_t = \begin{bmatrix} \text{current bit} \\ \text{previous bit} \end{bmatrix}$$

Two values are enough for this simple rule.

Here f is ReLU: replace negative values with 0.

The matrix products at t = 2

Current bit $x_2 = 1$. Previous state $h_1 = [1, 0]^T$.

Input contribution

W_{xh}

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix} \times \begin{bmatrix} 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Context contribution

W_{hh}

$$\begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

New state

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

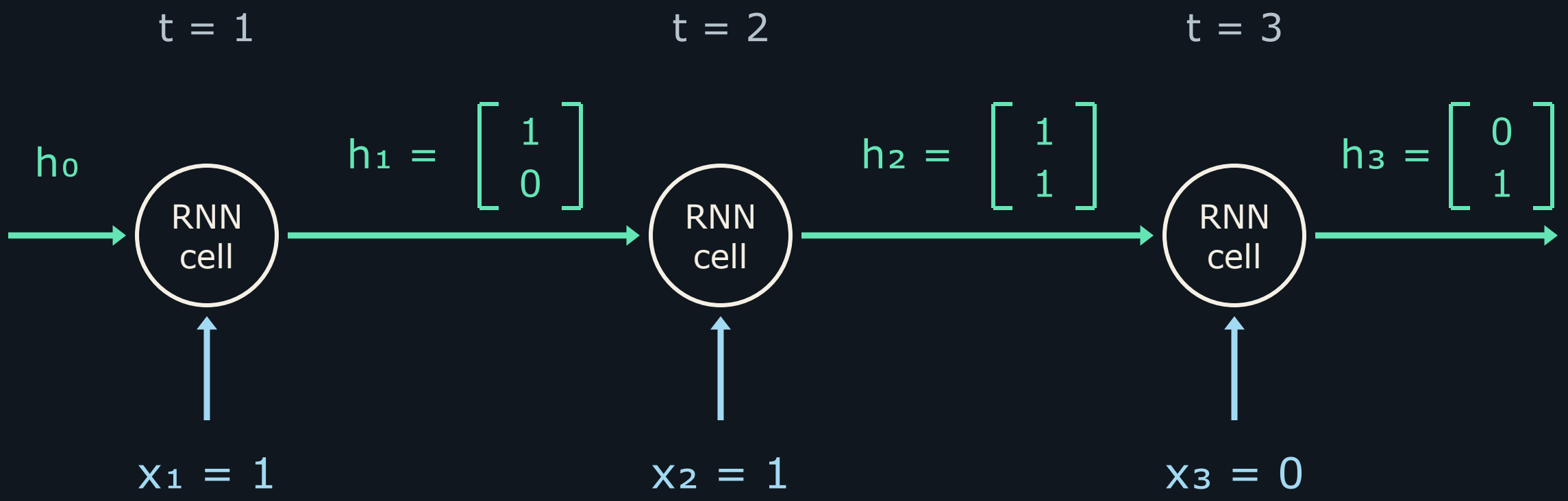
$h_2 = [\text{current, previous}]^T$

The bias is $[0, 0]^T$. ReLU keeps these nonnegative values unchanged.

W_{hh} copies the last current bit into the previous-bit slot.

One update at each time step

Input sequence: 1, 1, 0. Start with $h_0 = [0, 0]^T$.



The same weights perform all three updates.

Each step stores the current bit and the immediately preceding bit.

The output detects consecutive 1s

Output 1 only when both the current and previous bits are 1.

$$y_t = \text{ReLU}(W_{hy}\, h_t + b_y)$$

At t = 2

$$\underset{W_{hy}}{\begin{bmatrix} 1 & 1 \end{bmatrix}} \times \underset{h_2}{\begin{bmatrix} 1 \\ 1 \end{bmatrix}} \underset{\text{bias}}{- 1} = 1$$

$1 \times 1 + 1 \times 1 - 1 = 1$ **ReLU(1) = 1**

t = 1	**t = 2**	**t = 3**
$y_1 = 0$	$y_2 = 1$	$y_3 = 0$

A single 1 gives 0. Two consecutive 1s give 1.

Check: Follow the state through time

Choose one answer, then click to reveal the explanation.

At $t = 3$, $x_3 = 0$ and $h_2 = [1, 1]^T$.
What new state does our two-slot RNN produce?

A $[1, 1]^T$ because the state never changes.

B $[0, 0]^T$ because a zero input erases all context.

C $[0, 1]^T$ because it stores the current and previous bits.

D $[1, 0]^T$ because the first slot stores the previous bit.

Answer: C

At $t = 3$, the current bit is 0 and the previous bit is 1.

What if useful context is far away?

Useful information may need to survive many reading steps.



Earlier context can help with a later prediction.

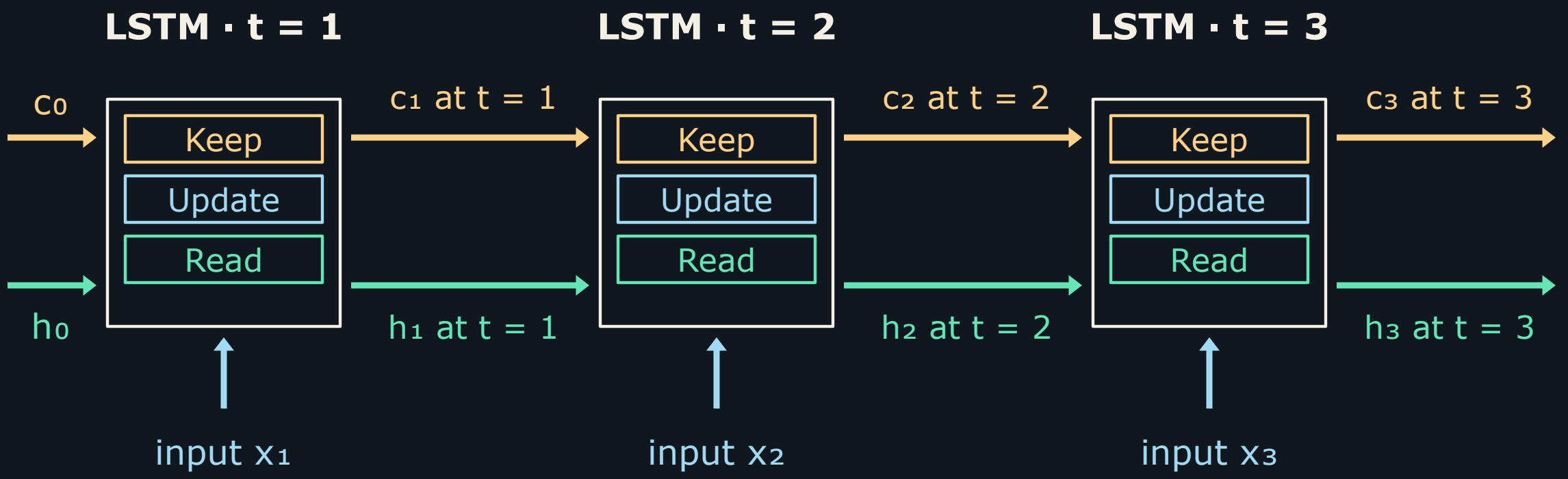
A basic RNN repeatedly updates its hidden state.

Useful earlier information can become harder to retain.

LSTM can help retain useful context across many steps.

LSTM: store, update, and read

Long short-term memory carries a memory state c and a hidden state h.



c: memory carried forward

h: current hidden state

Gates are learned controls for keeping, updating, and reading information.

Check: What does LSTM add?

Choose one answer, then click to reveal the explanation.

A basic RNN carries a hidden state h.
What does an LSTM add?

A Perfect storage of every past input forever.

B An extra cell state c, controlled by learned gates.

C A separate set of learned weights at every time step.

D A guarantee of better predictions on every task.

Answer: B

LSTM carries both c and h, with learned controls for managing context.
It can help with longer dependencies, but still needs training.